```
!pip install -q ultralytics torchvision pillow
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.7/46.7 kB 3.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 63.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.2/53.2 kB 3.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.7/88.7 kB 5.6 MB/s eta 0:00:00
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dr
```

```
import os
import shutil
from pathlib import Path

# Paths
DRIVE_UR_DATA = Path("/content/drive/MyDrive/Projects_DeepLearning/Research_Meta
OUTPUT_DIR = Path("/content/data")

TRAIN_FALL = OUTPUT_DIR / "train" / "fall"
TRAIN_NORMAL = OUTPUT_DIR / "train" / "normal"
VAL_FALL = OUTPUT_DIR / "val" / "fall"
VAL_NORMAL = OUTPUT_DIR / "val" / "normal"

for p in [TRAIN_FALL, TRAIN_NORMAL, VAL_FALL, VAL_NORMAL]:
    p.mkdir(parents=True, exist_ok=True)

# Validation sequences (held out subjects for testing generalization)
VAL_FALL_SEQS = ["fall-25", "fall-26", "fall-27", "fall-28", "fall-29", "fall-30
VAL_ADL_SEQS = ["adl-35", "adl-36", "adl-37", "adl-38", "adl-39", "adl-40"]

print(f"Scanning Drive folder: {DRIVE_UR_DATA}")

fall_count = 0
normal_count = 0

for file_path in DRIVE_UR_DATA.rglob("*"):
    if file_path.is_file() and file_path.suffix.lower() in [".png", ".jpg", ".jp
        full_path_str = str(file_path).lower()

        # Fall frames
        if "fall" in full_path_str:
            is_val = any(seq in full_path_str for seq in VAL_FALL_SEQS)
            dest = VAL_FALL if is_val else TRAIN_FALL
            dest_name = f"{file_path.parent.name}_{file_path.name}"
            shutil.copy2(file_path, dest / dest_name)
```

```
            fall_count += 1

        # ADL / Normal frames
        elif "adl" in full_path_str:
            is_val = any(seq in full_path_str for seq in VAL_ADL_SEQS)
            dest = VAL_NORMAL if is_val else TRAIN_NORMAL
            dest_name = f"{file_path.parent.name}_{file_path.name}"
            shutil.copy2(file_path, dest / dest_name)
            normal_count += 1

print("\nData Preparation Complete!")
print(f"Total Fall Frames Copied   : {fall_count}")
print(f"Total Normal Frames Copied : {normal_count}")
print(f"Dataset stored locally at  : {OUTPUT_DIR}")
```

```
Scanning Drive folder: /content/drive/MyDrive/Projects_DeepLearning/Research_Met

Data Preparation Complete!
Total Fall Frames Copied   : 2995
Total Normal Frames Copied : 8954
Dataset stored locally at  : /content/data
```

```
from pathlib import Path

# 1. Dataset path jo Step 1 me bana:
DATA_DIR = Path("/content/data")

# 2. Model weights ko sidha Drive par save karein (taaki runtime disconnect hor
MODEL_SAVE_PATH = Path("/content/drive/MyDrive/Projects_DeepLearning/Research_M
```

```
import os
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms
from PIL import Image
from pathlib import Path

DATA_DIR = Path("/content/data")
# Saving weights directly to your Google Drive folder
MODEL_SAVE_PATH = Path("/content/drive/MyDrive/Projects_DeepLearning/Research_Me

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Active Device: {device} ({torch.cuda.get_device_name(0) if torch.cuda.is_

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
```

```
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

class FallDataset(Dataset):
    def __init__(self, split_dir, transform=None):
        self.transform = transform
        # Label 0: FALL, Label 1: NORMAL
        fall_imgs = [(p, 0) for p in (split_dir / "fall").glob("*.*") if p.suffi
        normal_imgs = [(p, 1) for p in (split_dir / "normal").glob("*.*") if p.s

        self.num_falls = len(fall_imgs)
        self.num_normals = len(normal_imgs)
        self.samples = fall_imgs + normal_imgs
        print(f"[{split_dir.name}] Falls: {self.num_falls} | Normal: {self.num_n

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, label

def get_vit_model(num_classes=2):
    weights = models.ViT_B_16_Weights.DEFAULT
    model = models.vit_b_16(weights=weights)

    # Freeze lower layers
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze the last Transformer encoder layer for domain adaptation
    for param in model.encoder.layers[-1].parameters():
        param.requires_grad = True

    in_features = model.heads.head.in_features
    model.heads.head = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features, num_classes)
    )
    return model

# Initialize Datasets and Loaders
train_dataset = FallDataset(DATA_DIR / "train", transform=train_transform)
```

```
val_dataset = FallDataset(DATA_DIR / "val", transform=val_transform)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_worker
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, num_workers=2

# Class weight calculation to eliminate false fall alerts
total = train_dataset.num_falls + train_dataset.num_normals
weight_fall = total / (2.0 * train_dataset.num_falls)
weight_normal = total / (2.0 * train_dataset.num_normals)
class_weights = torch.tensor([weight_fall, weight_normal], dtype=torch.float).to
print(f"Computed Class Weights -> Fall: {weight_fall:.2f}, Normal: {weight_norma

model = get_vit_model(num_classes=2).to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights)
trainable_params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.AdamW(trainable_params, lr=1e-4, weight_decay=1e-4)

# Training loop
epochs = 2
for epoch in range(epochs):
    model.train()
    total_loss, correct, total_count = 0.0, 0, 0

    for i, (images, labels) in enumerate(train_loader, 1):
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        _, preds = torch.max(outputs, 1)
        correct += (preds == labels).sum().item()
        total_count += labels.size(0)

        if i % 10 == 0 or i == len(train_loader):
            print(f"Epoch [{epoch+1}/{epochs}] Step [{i}/{len(train_loader)}] Lo

    train_acc = 100.0 * correct / total_count

    # Validation evaluation
    model.eval()
    val_correct, val_total = 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            val_correct += (preds == labels).sum().item()
            val_total += labels.size(0)
```

```
    val_acc = 100.0 * val_correct / val_total if val_total > 0 else 0.0
    print(f"\n--- Epoch {epoch+1} Completed | Train Acc: {train_acc:.2f}% | Val

torch.save(model.state_dict(), MODEL_SAVE_PATH)
print("Model checkpoint saved to Google Drive:", MODEL_SAVE_PATH)
```

```
och [1/2] Step [110/299] Loss: 0.0002
och [1/2] Step [120/299] Loss: 0.0001
och [1/2] Step [130/299] Loss: 0.0001
och [1/2] Step [140/299] Loss: 0.0001
och [1/2] Step [150/299] Loss: 0.0001
och [1/2] Step [160/299] Loss: 0.0001
och [1/2] Step [170/299] Loss: 0.0001
och [1/2] Step [180/299] Loss: 0.0001
och [1/2] Step [190/299] Loss: 0.0001
och [1/2] Step [200/299] Loss: 0.0001
och [1/2] Step [210/299] Loss: 0.0001
och [1/2] Step [220/299] Loss: 0.0001
och [1/2] Step [230/299] Loss: 0.0001
och [1/2] Step [240/299] Loss: 0.0001
och [1/2] Step [250/299] Loss: 0.0001
och [1/2] Step [260/299] Loss: 0.0001
och [1/2] Step [270/299] Loss: 0.0001
och [1/2] Step [280/299] Loss: 0.0001
och [1/2] Step [290/299] Loss: 0.0001
och [1/2] Step [299/299] Loss: 0.0001

- Epoch 1 Completed | Train Acc: 99.32% | Val Acc: 100.00% ---

och [2/2] Step [10/299] Loss: 0.0000
och [2/2] Step [20/299] Loss: 0.0000
och [2/2] Step [30/299] Loss: 0.0000
och [2/2] Step [40/299] Loss: 0.0000
och [2/2] Step [50/299] Loss: 0.0001
och [2/2] Step [60/299] Loss: 0.0001
och [2/2] Step [70/299] Loss: 0.0001
och [2/2] Step [80/299] Loss: 0.0001
och [2/2] Step [90/299] Loss: 0.0001
och [2/2] Step [100/299] Loss: 0.0000
och [2/2] Step [110/299] Loss: 0.0001
och [2/2] Step [120/299] Loss: 0.0000
och [2/2] Step [130/299] Loss: 0.0000
```

```
och [2/2] Step [280/299] Loss: 0.0000
och [2/2] Step [290/299] Loss: 0.0000
och [2/2] Step [299/299] Loss: 0.0000

- Epoch 2 Completed | Train Acc: 100.00% | Val Acc: 100.00% ---

del checkpoint saved to Google Drive: /content/drive/MyDrive/Projects_DeepLearn
```

```
!pip install ultralytics
```

```
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist
Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.
Collecting ultralytics-thop>=2.1.6 (from ultralytics)
  Downloading ultralytics_thop-2.1.6-py3-none-any.whl.metadata (13 kB)
Collecting ultralytics-platform>=0.1.45 (from ultralytics)
  Downloading ultralytics_platform-0.1.57-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/d
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/di
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/d
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/pyt
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyth
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/pyt
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/di
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packag
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/pyt
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/pyth
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/py
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.13/dist
```

```
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-pac
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-pack
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/di
Downloading ultralytics-8.4.160-py3-none-any.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 32.3 MB/s eta 0:00:00
Downloading ultralytics_platform-0.1.57-py3-none-any.whl (89 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 89.9/89.9 kB 6.1 MB/s eta 0:00:00
Downloading ultralytics_thop-2.1.6-py3-none-any.whl (30 kB)
Installing collected packages: ultralytics-platform, ultralytics-thop, ultraly
Successfully installed ultralytics-8.4.160 ultralytics-platform-0.1.57 ultraly
```

```python
import re
import cv2
import torch
import torch.nn as nn
from torchvision import models, transforms
from PIL import Image
from pathlib import Path
from ultralytics import YOLO

# 1. Exact paths
FRAMES_DIR = Path("/content/drive/MyDrive/Projects_DeepLearning/Research_Metaver
OUTPUT_DIR = Path("/content/vit_inference_output")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_PATH = Path("/content/drive/MyDrive/Projects_DeepLearning/Research_Metaver
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# 2. Load fine-tuned ViT model
def load_vit_model():
    model = models.vit_b_16(weights=None)
    in_features = model.heads.head.in_features
    model.heads.head = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features, 2)
    )
    if not MODEL_PATH.exists():
        raise FileNotFoundError(f"Model checkpoint not found at: {MODEL_PATH}")
    model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
    model.to(device)
    model.eval()
    return model

vit_model = load_vit_model()
yolo = YOLO("yolov8n.pt")

vit_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
```

```
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

classes = ['FALL', 'NORMAL']

# Numerical sorting so frames process in 1, 2, ..., 466 sequence
def natural_sort_key(s):
    return [int(text) if text.isdigit() else text.lower() for text in re.split(r

image_paths = sorted([
    p for p in FRAMES_DIR.glob("*.*")
    if p.suffix.lower() in [".jpg", ".png", ".jpeg"]
], key=natural_sort_key)

fall_frames_count = 0
total_frames = len(image_paths)
print(f"Running inference on {total_frames} frames from: {FRAMES_DIR.name}...")

# 3. Frame-by-frame inference loop
for idx, img_path in enumerate(image_paths, 1):
    frame = cv2.imread(str(img_path))
    if frame is None:
        continue
    h, w, _ = frame.shape
    has_fall = False

    # Human detection (class 0: person)
    results = yolo(frame, classes=[0], verbose=False)

    if results and results[0].boxes:
        boxes = results[0].boxes.xyxy.cpu().numpy().astype(int)
        for box in boxes:
            x1, y1, x2, y2 = box
            x1, y1 = max(0, x1), max(0, y1)
            x2, y2 = min(w, x2), min(h, y2)

            # Filter noise crops
            if (x2 - x1) < 20 or (y2 - y1) < 20:
                continue

            crop = frame[y1:y2, x1:x2]
            pil_crop = Image.fromarray(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB))
            input_tensor = vit_transform(pil_crop).unsqueeze(0).to(device)

            with torch.no_grad():
                output = vit_model(input_tensor)
                probs = torch.softmax(output, dim=1)
                conf, pred_idx = torch.max(probs, 1)

            label = classes[pred_idx.item()]
            conf_val = conf.item() * 100
```

```
            # ViT decision threshold
            if label == 'FALL' and conf_val > 75.0:
                has_fall = True
                color = (0, 0, 255)
                tag = f"ViT FALL {conf_val:.1f}%"
            else:
                color = (0, 255, 0)
                tag = f"Normal {conf_val:.1f}%"

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
            cv2.putText(frame, tag, (x1, max(20, y1 - 8)), cv2.FONT_HERSHEY_SIMP

    if has_fall:
        fall_frames_count += 1
        cv2.rectangle(frame, (20, 20), (450, 75), (0, 0, 200), -1)
        cv2.putText(frame, "ALERT: PERSON FALL DETECTED", (30, 58), cv2.FONT_HER

    cv2.imwrite(str(OUTPUT_DIR / img_path.name), frame)

    if idx % 50 == 0 or idx == total_frames:
        print(f"Processed [{idx}/{total_frames}] frames... (Falls detected so fa

print("\n" + "="*50)
print(f"Inference Completed!")
print(f"Total Frames Processed : {total_frames}")
print(f"Frames with Fall Alert : {fall_frames_count}")
print(f"Annotated Frames Saved : {OUTPUT_DIR}")
print("="*50)
```

```
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=pat
Using device: cuda
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov
Running inference on 466 frames from: 07-00-46...
Processed [50/466] frames... (Falls detected so far: 0)
Processed [100/466] frames... (Falls detected so far: 3)
Processed [150/466] frames... (Falls detected so far: 3)
Processed [200/466] frames... (Falls detected so far: 3)
Processed [250/466] frames... (Falls detected so far: 3)
Processed [300/466] frames... (Falls detected so far: 3)
Processed [350/466] frames... (Falls detected so far: 3)
Processed [400/466] frames... (Falls detected so far: 3)
Processed [450/466] frames... (Falls detected so far: 3)
Processed [466/466] frames... (Falls detected so far: 3)

==================================================
Inference Completed!
Total Frames Processed : 466
Frames with Fall Alert : 3
Annotated Frames Saved : /content/vit_inference_output
==================================================
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.